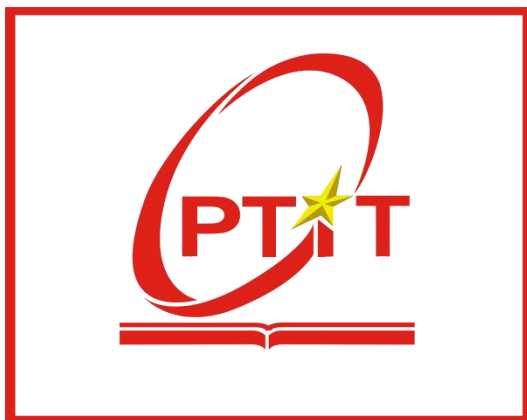


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN
MÔN LẬP TRÌNH PYTHON

Giảng viên : Kim Ngọc Bách

Lớp : B24CQCE02-B

Nhóm : 02

Sinh viên : Nguyễn Thị Hồng Hải - Lê Văn Khoa

Hà Nội , tháng 5/2026

MỤC LỤC

I.TỔNG QUAN	
II.MODULE 1-WEB SCRAPING.....	
2.1. Mục Tiêu.....	
2.2. Thư Viện Sử Dụng.....	
2.3. Thuật Toán và Logic Xử Lý.....	
2.4. Xử lý Ngoại Lệ.....	
2.5. Kết Quả Đạt Được.....	
III. MODULE 2 - FLASK REST API.....	
3.1. Mục Tiêu.....	
3.2. Kiến Trúc API và Thư Viện Sử Dụng.....	
3.3. Phân Tích Từng Hàm.....	
3.4. Mã Trạng Thái HTTP.....	
3.5. Kết Quả Đạt Được.....	
IV. MODULE 3 - GUI DESKTOP.....	
4.1. Mục Tiêu.....	
4.2. Thư Viện Sử Dụng.....	
4.3. Kiến Trúc Lớp PlayerCompareApp.....	
4.4. Thuật Toán Radar Chart.....	
4.5. Kết Quả Đạt Được.....	
V.MODULE 4 - PHÂN TÍCH THỐNG KÊ ĐỘI BÓNG.....	
5.1. Mục Tiêu.....	
5.2. Các Chỉ Số Phân Tích và Thư Viện Sử Dụng.....	
5.3. Phân Tích Hàm analyze_and_find_best_team().....	
5.4. Kết Quả Đạt Được.....	
VI.MODULE 5 - PHÂN CỤM K-MEANS VÀ PCA.....	
6.1. Mục Tiêu.....	
6.2. Pipeline Xử Lý và Thư Viện Sử Dụng.....	
6.3. Tiền Xử Lý Dữ Liệu.....	
6.4. Thuật Toán K-Means.....	
6.5. Thuật Toán PCA.....	
6.6. Trực Quan Hóa Kết Quả.....	
6.7. Ý Nghĩa 4 Cụm Cầu Thủ.....	
6.8. Kết Quả Đạt Được.....	
VII. KẾT LUẬN.....	
7.1. Tổng Kết.....	
7.2. Công Nghệ Sử Dụng.....	

I. TỔNG QUAN

Báo cáo được xây dựng nhằm thu thập, lưu trữ và phân tích toàn diện dữ liệu thống kê cầu thủ tại giải Ngoại Hạng Anh mùa 2025-2026 từ website FBRef.com. Hệ thống được thiết kế theo kiến trúc pipeline gồm 5 module độc lập, liên kết với nhau theo luồng dữ liệu:

Module	Chức năng
1	Web Scraping - Thu thập dữ liệu từ FBRef bằng Selenium
2	FLASK REST API - Cung cấp dữ liệu qua Flask HTTP API
3	GUI Desktop - So sánh trực quan 2 cầu thủ bằng Radar Chart
4	Phân tích thống kê - Tính toán chỉ số đội bóng tốt nhất
5	Machine Learning - Phân cụm cầu thủ với K-Means + PCA

II. MODULE 1-WEB SCRAPING

2.1. Mục Tiêu

Thu thập dữ liệu thống kê phong trào (Miscellaneous Stats) của các cầu thủ thi đấu trên 90 phút tại Premier League năm 2025-2026 từ trang FBRef.com, lưu kết quả vào file CSV.

2.2. Thư Viện Sử Dụng

Thư Viện	Mục Đích Sử Dụng
pandas	Đọc bảng HTML, xử lý DataFrame, lọc dữ liệu, xuất CSV
io.StringIO	Chuyển chuỗi HTML sang luồng dữ liệu để <code>pd.read_html()</code> đọc được
undetected_chromedriver (uc)	Khởi động Chrome ẩn danh, vượt qua cơ

	chế chống bot của FBRef
selenium (WebDriverWait, EC, By)	-Điều khiển trình duyệt, chờ phần tử DOM(mô hình đối tượng tài liệu) xuất hiện. -By:định danh cách muốn tìm một phần tử trên trang web. -EC: định nghĩa điều kiện mong đợi - WebDriverWait: điều khiển thời gian chờ đợi.

2.3. Thuật Toán và Logic Xử Lý

Bước 1 - Khởi tạo trình duyệt

`options = uc.ChromeOptions()` : tạo ra một đối tượng chứa các cấu hình mặc định của Chrome, có thể thêm các tùy chỉnh vào đối tượng này để thay đổi hành vi của trình duyệt.

`driver = uc.Chrome(options=options, version_main=147)`: Sử dụng `undetected_chromedriver` thay vì Selenium thông thường để tránh bị phát hiện là bot. Tham số `version_main=147` khớp với phiên bản Chrome đã cài trên máy.

Bước 2 - Truy cập và chờ bảng dữ liệu

`url = 'https://fbref.com/en/comps/9/misc/Premier-League-Stats'`
`driver.get(url)`
 : truy cập vào một địa chỉ trang web cụ thể.

`wait = WebDriverWait(driver, 25)`
`table_element = wait.until(EC.presence_of_element_located((By.ID, 'stats_misc')))` : WebDriverWait chờ tối đa 25 giây cho đến khi phần tử có `id='stats_misc'` xuất hiện trong DOM - đây là bảng thống kê phong trào chính thức của FBRef.

Bước 3 - Đọc bảng HTML vào DataFrame

Lấy `outerHTML` của phần tử bảng đó , dùng `pd.read_html()` để chuyển sang DataFrame. Tiếp theo dùng `MultiIndex columns` (cột cha/con) để kiểm tra xem tiêu đề của bảng có đang ở dạng nhiều tầng hay không, nếu có thì cần lấy level cuối cùng để lấy tên cột thực sự.

`html_content = table_element.get_attribute('outerHTML')`
`df_list = pd.read_html(io.StringIO(html_content))`
`df = df_list[0]`
`if isinstance(df.columns, pd.MultiIndex):`
 `df.columns = df.columns.get_level_values(df.columns.nlevels - 1)`

Bước 4 - Làm sạch và lọc dữ liệu

Đầu tiên là làm sạch tên cột (loại bỏ khoảng trắng dư thừa). Xóa cột đầu tiên (cột số thứ tự của FBRef). Chuẩn hóa cột 'Nation' bằng cách chỉ giữ mã quốc gia (phần tử cuối khi split). Chuyển đổi kiểu dữ liệu của cột chứa số phút thi đấu từ dạng văn bản (String) sang dạng số (Numeric). Lọc cầu thủ có số phút thi đấu > 90 để đảm bảo có ít nhất 1 trận đầy đủ. Cuối cùng là loại bỏ các header lặp lại.

```
df = df.iloc[:, 1:]
df['Nation'] = df['Nation'].str.split().str[-1]
df[col_min] = pd.to_numeric(df[col_min], errors='coerce')
df = df[df[col_min] > 90].copy()
df[df['Player'] != 'Player']
```

Bước 5 - Xuất file CSV

Điền 'N/a' cho các ô trống, đặt lại chỉ số, thêm cột STT, lưu file với encoding UTF-8 BOM (utf-8-sig) để Excel đọc đúng tiếng Việt.

```
df = df.fillna('N/a')
df.reset_index(drop=True, inplace=True)
df.insert(0, 'STT', range(1, len(df) + 1))
file_name = "cầu thủ thi đấu trên 90ph.csv"
df.to_csv('cầu thủ thi đấu trên 90ph.csv', index=False, encoding='utf-8-sig')
```

2.4. Xử Lý Ngoại Lệ

- Dùng try/except/finally bao toàn bộ logic - đảm bảo driver.quit() luôn được gọi kể cả khi có lỗi.
- errors='coerce' trong pd.to_numeric(): nếu giá trị không phải số sẽ trở thành NaN.
- Kiểm tra tồn tại cột 'Min' hoặc 'Minutes' hoặc '90s' để tăng tính linh hoạt với cấu trúc bảng thay đổi.

2.5. Kết Quả Đạt Được

- File 'cầu thủ thi đấu trên 90ph.csv' chứa dữ liệu của tất cả cầu thủ có trên 90 phút thi đấu.
- Dữ liệu bao gồm: tên, quốc tịch, đội bóng, số thẻ, lỗi, phá bóng, tắc bóng,...
- File được mã hóa UTF-8 BOM để tương thích với Microsoft Excel.

III. MODULE 2 - FLASK REST API

3.1. Mục Tiêu

Xây dựng một REST API đơn giản bằng flask để phục vụ dữ liệu cầu thủ từ file CSV cho các ứng dụng client như giao diện GUI ở Module 3.

3.2. Kiến Trúc API Và Thư Viện Sử Dụng

Endpoint	Method	Tham số	Mô tả
/api/player	GET	?name=<ten>	Lấy thông tin chi tiết của 1 cầu thủ

			theo tên
/api/players/all	GET	Không có	Lấy danh sách tên tất cả cầu thủ trong dataset

Thư Viện	Mục Đích Sử Dụng
flask(Flask,jsonify,request)	- để tạo website và API -Flask:một framework cực kỳ phổ biến để tạo website và API -jsonify:biến dữ liệu (như danh sách hoặc từ điển Python) thành định dạng JSON->định dạng tiêu chuẩn để các ứng dụng "nói chuyện" với nhau qua internet. -request:xử lý dữ liệu mà người dùng gửi lên server
os	để tương tác với hệ điều hành
pandas	Đọc bảng HTML, xử lý DataFrame, lọc dữ liệu, xuất CSV

3.3. Phân Tích Từng Hàm

`app = Flask(__name__)` : Khởi tạo ứng dụng Flask và biến này sẽ quản lý toàn bộ các đường dẫn và xử lý yêu cầu web.

Hàm `load_data()`

Hàm tiện ích kiểm tra file CSV tồn tại rồi đọc vào DataFrame. Trả về None nếu file không tồn tại, giúp các endpoint xử lý trường hợp chưa có dữ liệu.

```
def load_data():
    if os.path.exists(CSV_FILE):
        return pd.read_csv(CSV_FILE)
    return None
```

Hàm `get_player_stats()`

Nhận tham số 'name' từ query string. Kiểm tra xem dữ liệu có nạp thành công không. Tìm kiếm không phân biệt hoa thường bằng `.str.lower()`. Sau đó,chuyển đổi bảng dữ liệu thành một danh sách các từ điển. Cuối cùng, trả về object JSON chứa toàn bộ dữ liệu cầu thủ hoặc thông báo lỗi phù hợp (400/404/500).

```
player_name = request.args.get('name')
df=load_data()
result = df[df['Player'].str.lower() == player_name.strip().lower()]
```

```
player_data = result.to_dict(orient='records')[0]
return jsonify({'status': 'success', 'data': player_data})
```

Hàm get_all_player()

Trả về danh sách tên tất cả cầu thủ kèm tổng số. Dùng .tolist() để chuyển Series sang list Python trước khi jsonify.

```
players = df['Player'].tolist()
return jsonify({'total': len(players), 'players': players})
```

3.4. Mã Trạng Thái HTTP

Mã	Tình Huống
200 OK	Tìm thấy cầu thủ, trả về dữ liệu thành công
400 Bad Request	Không cung cấp tham số 'name' trong request
404 Not Found	Tên cầu thủ không tồn tại trong dataset
500 Internal Server Error	Không tìm thấy file CSV dữ liệu

3.5 . Kết Quả Đạt Được

- API chạy local tại http://127.0.0.1:5000, dễ dàng mở rộng lên server thực.
- Tách biệt lớp dữ liệu và lớp hiển thị - Module 3 chỉ cần gọi HTTP, không cần đọc CSV trực tiếp.
- Hỗ trợ tìm kiếm case-insensitive tiện lợi cho người dùng.

IV. MODULE 3 - GUI DESKTOP

4.1. Mục Tiêu

Xây dựng ứng dụng desktop bằng Tkinter cho phép người dùng tìm kiếm, xem và so sánh trực quan các chỉ số của 2 cầu thủ thông qua biểu đồ Radar Chart.

4.2. Thư Viện Sử Dụng

Thư Viện	Mục Đích Sử Dụng
tkinter(tk,messagebox,ttk)	-tk:Framework GUI chính - tạo cửa sổ, nhãn, ô nhập, nút bấm -messagebox: Dùng để hiển thị các hộp thoại thông báo (như báo lỗi, xác nhận yes/no). -ttk:cung cấp các thành phần giao diện (widgets) có thiết kế hiện đại và đẹp mắt

	hơn so với các widget cơ bản của Tkinter.
requests	Gọi HTTP đến Flask API để lấy dữ liệu cầu thủ
matplotlib (pyplot + FigureCanvasTkAgg)	Vẽ Radar Chart và nhúng vào cửa sổ Tkinter
numpy	Tính toán góc (angles) cho các trục của Radar Chart

4.3 Kiến Trúc Lớp PlayerCompareApp

Hàm `__init__()`

Khởi tạo cửa sổ chính (900x700px), đặt URL API, khởi tạo mảng `players_data=[None, None]` lưu dữ liệu 2 cầu thủ, định nghĩa danh sách 10 chỉ số theo dõi. Gọi `create_widgets()` để dựng giao diện.

```
self.root.geometry("900x700")
self.api_url="http://127.0.0.1:5000/api/player"
self.stats_columns=['90s','CrdY','CrdR','Fls','Fld','Off','Crs','Int','TklW','OG']
self.players_data = [None, None]
self.check_vars = {}
self.create_widgets()
```

Hàm `create_widgets()`

Dùng Grid layout để sắp xếp các widget nhập tên, nút tìm kiếm cho cả 2 cầu thủ trên cùng một hàng. Tạo frame kết quả và nút 'So sánh' màu xanh nổi bật.

```
tk.Label(input_frame, text="Player 1:").grid(row=0, column=0)
self.ent_p1 = tk.Entry(input_frame)
self.ent_p1.grid(row=0, column=1, padx=5)
tk.Button(input_frame, text="Search", command=lambda:
self.search_player(0)).grid(row=0, column=2)
tk.Label(input_frame, text="Player 2:").grid(row=0, column=3, padx=(20, 0))
self.ent_p2 = tk.Entry(input_frame)
tk.Button(input_frame, text="Search", command=lambda:
self.search_player(1)).grid(row=0, column=5)
self.result_frame = tk.Frame(self.root)
self.result_frame.pack(fill=tk.BOTH, expand=True, padx=20)
self.stats_frame = tk.Frame(self.result_frame)
```



```
tk.Button(self.root, text="So sánh / Compare", bg="green",
fg="white",font=('Arial', 12, 'bold'), command=self.draw_radar_chart).pack(pady=10)
```

Hàm search_player()

Gọi GET /api/player?name=<tên> qua requests.get(). Nếu thành công (status 200), lưu JSON data vào players_data[idx] và refresh bảng hiển thị. Xử lý lỗi kết nối qua messagebox.

```
response = requests.get(self.api_url, params={'name': name})
if response.status_code == 200:
    self.players_data[idx] = response.json()['data']
    self.display_stats()
else:
    messagebox.showerror("Lỗi", response.json().get('message', 'Không tìm thấy'))
```

Hàm display_stats()

Xóa toàn bộ widget cũ trong stats_frame rồi vẽ lại bảng so sánh. Mỗi hàng có 1 Checkbutton để chọn/bỏ chỉ số, giá trị cầu thủ 1 và cầu thủ 2. Dùng tk.BooleanVar() để lưu trạng thái checkbox. Tạo ô tích chọn. Truy cập vào dữ liệu của từng cầu thủ để lấy giá trị của chỉ số đó. Nếu không tìm thấy chỉ số, hiện "N/a".Cuối cùng đưa các chỉ số lên các cột tương ứng.

```
for widget in self.stats_frame.winfo_children():
    widget.destroy()

tk.Label(self.stats_frame, text="Chỉ số", font=('Arial', 10, 'bold')).grid(row=0,
column=0, sticky='w')
tk.Label(self.stats_frame, text="Cầu thủ 1", font=('Arial', 10,
'bold')).grid(row=0, column=1, padx=20)
tk.Label(self.stats_frame, text="Cầu thủ 2", font=('Arial', 10,
'bold')).grid(row=0, column=2, padx=20)
for i, col in enumerate(self.stats_columns):
    if col not in self.check_vars:
        self.check_vars[col] = tk.BooleanVar(value=True)
    tk.Checkbutton(self.stats_frame, text=col,
variable=self.check_vars[col]).grid(row=i + 1, column=0,sticky='w')
    val1 = self.players_data[0].get(col, "N/a") if self.players_data[0] else "-"
    tk.Label(self.stats_frame, text=val1).grid(row=i + 1, column=1)
    val2 = self.players_data[1].get(col, "N/a") if self.players_data[1] else "-"
    tk.Label(self.stats_frame, text=val2).grid(row=i + 1, column=2)
```

Hàm draw_radar_chart()

Đây là phương thức cốt lõi nhất - vẽ biểu đồ Radar Chart để so sánh trực quan:

1. Lấy các chỉ số được chọn (checkbox = True)

```
selected_stats = [s for s in self.stats_columns if self.check_vars[s].get()]
```

2. Tính góc đều nhau cho các trục (0 -> 2π)

```
angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False).tolist()
```

3. Nối điểm đầu vào cuối để đóng đa giác

```
values1 += values1[:1]
```

```
values2 += values2[:1]
```

```
angles += angles[:1]
```

4. Vẽ trên Axes polar (tọa độ cực)

```
fig, ax = plt.subplots(figsize=(5,5), subplot_kw=dict(polar=True))
```

```
ax.fill(angles, values1, color='red', alpha=0.25)
```

```
ax.plot(angles, values1, color='red', linewidth=2,
```

```
label=self.players_data[0]['Player'])
```

```
ax.fill(angles, values2, color='blue', alpha=0.25)
```

```
ax.plot(angles, values2, color='blue', linewidth=2,
```

```
label=self.players_data[1]['Player'])
```

4.4. Thuật Toán Radar Chart

- Dùng tọa độ cực (polar coordinates): góc xác định chỉ số, bán kính xác định giá trị.
- `np.linspace(0, 2*pi, n)` chia đều vòng tròn thành n góc bằng nhau cho n chỉ số.
- Nối phần tử đầu vào cuối list (`values[:1]`, `angles[:1]`) để đóng kín đa giác.
- `fill()` tô màu diện tích, `plot()` vẽ đường viền, `alpha=0.25` tạo độ trong suốt.
- Biểu đồ được nhúng vào Toplevel window qua `FigureCanvasTkAgg`.

4.5. Kết Quả Đạt Được

- Giao diện trực quan, dễ sử dụng cho người không biết code.
- Cho phép lựa chọn linh hoạt các chỉ số cần so sánh qua checkbox.
- Radar Chart giúp nhận biết nhanh điểm mạnh/yếu của từng cầu thủ.

V. MODULE 4 - PHÂN TÍCH THỐNG KÊ ĐỘI BÓNG

5.1. Mục Tiêu

Tính toán các chỉ số thống kê tổng hợp (median, mean, std) theo từng đội bóng, xác định đội dẫn đầu từng hạng mục và kết luận đội có phong độ tốt nhất.

5.2. Các Chỉ Số Phân Tích Và Thư Viện Sử Dụng

Chỉ Số	Ý Nghĩa	Loại
90s	Số trận 90 phút đã chơi	Tích cực
CrdY	Thẻ vàng	Tiêu cực
CrdR	Thẻ đỏ	Tiêu cực
2CrdY	2 thẻ vàng = thẻ đỏ	Tiêu cực
Fls	Số lần phạm lỗi	Tiêu cực
Fld	Số lần bị phạm lỗi (gây ra phạt đền)	Tiêu cực
Off	Việt vị	Tiêu cực
Crs	Đường tạt bóng	Tích cực
Int	Số lần cắt bóng (intercept)	Tích cực
TkLW	Số pha tắc bóng thắng	Tích cực
PKwon	Penalty kiếm được	Tích cực
PKcon	Penalty để đối phương ghi bàn	Tiêu cực
OG	Bàn phản lưới	Tiêu cực

Thư Viện	Mục Đích Sử Dụng
pandas	Đọc bảng HTML, xử lý DataFrame, lọc dữ liệu, xuất CSV
numpy	tính toán toán học, xử lý mảng (array) dữ liệu lớn và các phép toán ma trận rất nhanh chóng

5.3. Phân Tích Hàm `analyze_and_find_best_team()`

Bước 1- Đọc và chuẩn hóa dữ liệu

Chuyển tất cả cột số sang kiểu float với `errors='coerce'` (giá trị lỗi -> NaN) rồi `fillna(0)` để tránh ảnh hưởng tính toán thống kê.

for col in numeric_cols:

```
if col in df.columns:  
    df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)
```

Bước 2 - Tổng hợp thống kê theo đội

Dùng groupby('Squad') để nhóm các cầu thủ theo đội bóng, sau đó agg(['median', 'mean', 'std']) tính đồng thời cả 3 chỉ số thống kê. MultiIndex columns được flatten thành dạng 'chỉ_số_thống_kê' (vd: '90s_mean').

```
team_stats = df.groupby('Squad')[numeric_cols].agg(['median', 'mean', 'std'])  
team_stats.columns = [f'{stat}_{metric}' for stat, metric in team_stats.columns]
```

Bước 3 - Tìm đội dẫn đầu từng chỉ số

Với mỗi chỉ số, dùng idxmax() để lấy tên đội có giá trị mean cao nhất. Chỉ đếm điểm cho các chỉ số tích cực (positive_stats) - đội dẫn đầu chỉ số tiêu cực sẽ bị bỏ qua.

```
positive_stats = ['90s', 'Fld', 'Crs', 'Int', 'TklW', 'PKwon']  
for col in numeric_cols:  
    mean_col = f'{col}_mean'  
    top_team = team_stats[mean_col].idxmax()  
    if col in positive_stats:  
        leader_counts[top_team] = leader_counts.get(top_team, 0) + 1
```

Bước 4 - Kết luận đội tốt nhất

Đội nào dẫn đầu nhiều chỉ số tích cực nhất sẽ được chọn là đội có phong độ tốt nhất mùa giải.

```
best_team = max(leader_counts, key=leader_counts.get)  
max_leads=leader_counts[best_team]
```

5.4. Kết Quả Đạt Được

- File CSV 'thống kê median mean std của các đội bóng.csv' chứa bảng thống kê đầy đủ.
- In chi tiết đội dẫn đầu từng chỉ số cùng giá trị cụ thể.
- Kết luận rõ ràng đội bóng có phong độ tốt nhất dựa trên bảng chứng định lượng.

VI. MODULE 5 - PHÂN CỤM K-MEANS VÀ PCA

6.1. Mục Tiêu

Áp dụng thuật toán học máy không giám sát K-Means để phân nhóm cầu thủ dựa trên phong cách chơi. Dùng PCA giảm chiều dữ liệu để trực quan hóa các cụm trên không gian 2D và 3D.

6.2. Pipeline Xử Lý

Bước	Kỹ Thuật	Mục Đích
Chuẩn hóa	StandardScaler	Đưa tất cả features về cùng thang đo (mean=0, std=1)
Tìm k tối ưu	Elbow Method + Silhouette Score	Xác định số cụm hợp lý nhất
Phân cụm	K-Means (k=4)	Gán nhãn cluster cho từng cầu thủ
Giảm chiều 2D	PCA (n_components=2)	Giữ 2 chiều chính để vẽ scatter plot 2D
Giảm chiều 3D	PCA (n_components=3)	Giữ 3 chiều chính để vẽ scatter plot 3D

Thư Viện	Mục Đích Sử Dụng
pandas	Đọc bảng HTML, xử lý DataFrame, lọc dữ liệu, xuất CSV
numpy	tính toán toán học, xử lý mảng (array) dữ liệu lớn và các phép toán ma trận rất nhanh chóng
matplotlib	vẽ biểu đồ (2D, biểu đồ cột, đường thẳng...)
sklearn(KMeans,StandardScaler,silhouette_score,PCA)	-KMeans: phân cụm dữ liệu bằng cách tách các mẫu thành n nhóm -StandardScaler: dùng để chuẩn hóa dữ liệu -silhouette_score:Hàm tính điểm Silhouette. Đây là chỉ số đo lường chất lượng phân cụm (điểm càng cao, các cụm càng tách biệt và rõ ràng). -PCA: Phân tích thành phần chính
mpl_toolkits(Axes3D)	hỗ trợ vẽ biểu đồ không gian 3 chiều (3D) trong Matplotlib.

6.3 Tiền Xử Lý Dữ Liệu

Dữ liệu thô có thang đo rất khác nhau: '90s' có thể từ 0-38, trong khi 'CrdR' thường là 0-2. Nếu không chuẩn hóa, K-Means sẽ bị chi phối bởi các cột có giá trị lớn hơn. StandardScaler chuyển mỗi cột thành phân phối chuẩn $N(0,1)$:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

6.4. Thuật Toán K-Means

Nguyên Lý Hoạt Động

Khởi tạo ngẫu nhiên k tâm cụm (centroids). Gán mỗi điểm dữ liệu vào cụm có centroid gần nhất (khoảng cách Euclidean). Cập nhật centroid = trung bình của tất cả điểm trong cụm. Lặp lại cho đến khi centroid không đổi (hội tụ).

```
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)
```

Phương pháp Elbow để chọn k

Vẽ đồ thị inertia (tổng bình phương khoảng cách đến centroid) theo k từ 2-10. 'Điểm khuỷu tay' (elbow) là nơi inertia giảm chậm lại đột ngột - đó là k tối ưu.

```
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
    silhouette_avg.append(silhouette_score(X_scaled, kmeans.labels_))
```

Silhouette Score xác nhận k

Silhouette Score đo mức độ phù hợp của điểm dữ liệu trong cụm của nó so với cụm kế cận (range -1 đến +1, càng cao càng tốt). Kết hợp 2 phương pháp giúp chọn k=4 là số cụm hợp lý.

6.5. Thuật toán PCA (Principal Component Analysis)

PCA giảm chiều dữ liệu từ 13 features xuống 2 hoặc 3 chiều bằng cách tìm các trục có phương sai lớn nhất trong dữ liệu. Các trục này gọi là Principal Components (PC).

```
pca_2d = PCA(n_components=2)
X_pca_2d = pca_2d.fit_transform(X_scaled)

pca_3d = PCA(n_components=3)
X_pca_3d = pca_3d.fit_transform(X_scaled)
```

6.6. Trực quan hóa kết quả

Biểu đồ Elbow + Silhouette (elbow_silhouette.png)

- Subplot trái: đường cong inertia giảm dần - tìm điểm khuỷu tay.
- Subplot phải: đường silhouette score - tìm đỉnh cao nhất.

- Lưu file PNG bằng plt.savefig() để chia sẻ/báo cáo.

Scatter Plot PCA 2D (pca_2d.png)

Vẽ scatter plot với trục X=PC1, trục Y=PC2, màu sắc theo cluster. Dùng colorbar để hiển thị nhãn cụm.

```
plt.scatter(X_pca_2d[:,0], X_pca_2d[:,1], c=df['Cluster'], cmap='viridis',
alpha=0.7)
```

Scatter Plot PCA 3D (pca_3d.png)

Dùng mpl_toolkits.mplot3d để vẽ scatter 3D với 3 Principal Components trên 3 trục không gian.

```
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X_pca_3d[:,0], X_pca_3d[:,1], X_pca_3d[:,2], c=df['Cluster'],
cmap='viridis')
```

6.7. Ý nghĩa 4 cụm cầu thủ

Cụm	Mô Tả Dự Kiến	Đặc Điểm Chính
Cluster 0	Cầu thủ phòng thủ chuyên nghiệp	Int,TkLW cao; Crs,Fld ít
Cluster 1	Cầu thủ tấn công sáng tạo	Crs,Fld cao; ít CrdY,Fls
Cluster 2	Cầu thủ cứng rắn/ hung hăng	CrdY, Fls cao; TkLW trung bình
Cluster 3	Cầu thủ dự bị/ít thi đấu	90s thấp ; các chỉ số đều thấp

6.8. Kết Quả Đạt Được

- 3 file ảnh PNG: elbow_silhouette.png, pca_2d.png, pca_3d.png.
- Cột 'Cluster' được thêm vào DataFrame phân loại từng cầu thủ vào 4 nhóm.
- Cung cấp góc nhìn khách quan về phong cách chơi dựa trên số liệu thực tế.

VII. Kết Luận

7.1. Tổng Kết

Báo cáo đã xây dựng thành công một pipeline phân tích dữ liệu bóng đá hoàn chỉnh, từ thu thập dữ liệu thô đến trực quan hóa và phân tích học máy:

Tiêu Chí	Đánh Giá
Thu thập dữ liệu tự động	Selenium/undetected_chromedriver vượt qua anti-bot thành công
Tính linh hoạt hệ thống	API phục vụ nhiều client khác nhau (GUI, web, mobile)
Trực quan hóa	Radar Chart và PCA giúp nhận diện trực quan điểm mạnh/yếu
Phân tích thống kê	Kết luận khách quan dựa trên median/mean/std
Machine Learning	K-Means + Silhouette Score cho phân cụm có căn cứ khoa học

7.2. Công Nghệ Sử Dụng

Nhóm	Công Nghệ
Web Scraping	Selenium, undetected_chromedriver, BeautifulSoup (qua pd.read_html)
Backend / API	Flask, pandas, os
Frontend / GUI	Tkinter, matplotlib, FigureCanvasTkAgg
Thống kê	pandas groupby/agg, numpy
Machine Learning	scikit-learn (KMeans, StandardScaler, PCA, silhouette_score)
Lưu trữ	CSV (pandas to_csv, read_csv)